OOP using C++ Programming

Trainer : Priyanka R. Rangole

Email: priyanka.rangole@sunbeaminfo.com

Constant in C++

- We can declare a constant variable that cannot be modified in the app.
- If we do not want to modify the value of the variable then const keyword is used.
- A constant variable is also called a read only variable.
- The value of such a variable should be known at compile time.
- In C++ , Initializing constant variable is mandatory
- const int i=3; //VALID
- Const int val; //Not ok in c++
- Generally const keyword is used with the argument of function to ensure that the variable cannot be

  modified within that function.

Constant data member

- Once initialized, if we do not want to modify the state of the data member inside any member function of the class including the constructor body then we should declare the data member constant.
- If we declare data member constant then it is mandatory to initialize it using constructors member initializer list.

Const member function

- The member function can be declared as const. In that case the object invoking the function cannot be modified within that member function.

- We can not declare global function constant but we can declare member function constant.

- If we do not want to modify state of current object inside member function then we should declare members function as constant.

- void display() const; , void printData() const;

- Even though normal members cannot be modified in const function, but *mutable* data members are allowed to modify.

Const member function

- In constant member function, if we want to modify state of non constant data member then we

  should

  use **mutable keyword.**

- We can not declare following function constant:
  1. Global Function
  2. Static Member Function
  3. Constructor
  4. Destructor

Const object

- If we don’t want to modify the state of the object then we should declare the object constant.
- On non constant object, we can call constant as well as non constant member function.
- On a Constant object, we can call only constant member functions

Reference

- Reference is derived data type.

- It is an alias or another name given to the existing memory location / object.
  - Example : int a=10; int &r = a;
  - In the above example a is referent variable and r is reference variable.
  - It is mandatory to initialize reference.

- Reference is alias to a variable and cannot be reinitialized to other variable

- When '&' operator is used with reference, it gives the address of the variable to which it refers.

- Reference can be used as data member of any class

Reference

- We can not create reference to constant value.
  - int &num2 = 10; //can not create reference to constant value
- Reference is internally considered as constant pointer hence referent of reference must be variable/object.

```
int main( void )
{
        int num1 = 10;
        int &num2 = num1;
        cout<<"Num2 :

        "<<num2<<endl; return 0;


}
```

Difference between Pointers and reference

Pointer

- It is a variable that points to another variable.
- To access the value of a variable with the help of a pointer, we need to do dereferencing explicitly.
- We can create a pointer to pointer
- We can create a pointer without initialization. Create a NULL pointer.

Eg int n=5; int* ptr=&n;

Reference

- It is an alias / secondary name to an already existing memory.
- No need of dereferencing to access a value of a variable with ref.
- We can't create a reference to reference.
- We can't create a ref without initialization NULL ref can't be created.

Eg : int n=5; int& ref=n;

pass arguments to function, by value, by address or by reference.

- **In C++, we can pass argument to the function using 3 ways:**

1. By Value
2. By Address
3. By Reference

- If a variable is passed by reference, then any change made in the variable within the function is reflected in the caller function.

- Reference can be argument or return type of any function

Dynamic Memory Allocation

To allocate memory dynamically then we should use a new operator
To deallocate that memory we should use **delete** operator.
Dangling pointer :- The pointer which contains, address of deallocated memory.
Memory leakage :- When we allocate space in memory, and if we loose pointer to reach that memory then such wastage of memory is called memory leakage.

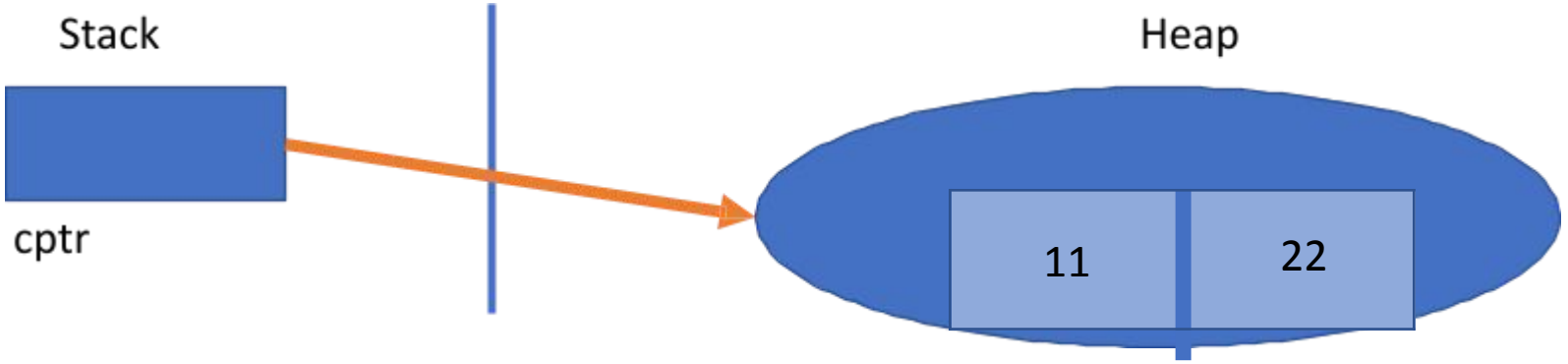
```
int main()

{
                                        // allocate memory
        cout<<"Value:"<<*ptr<<endl; //Dereferencing
        int *ptr = new int;             //Dereferencing
        delete ptr;    // deallocate memory
        *ptr = 125;
        ptr = NULL;
        return 0;
}
```

Heap Based object

```
Complex *cptr=new Complex (11,22) ;
delete cptr;
```

- By using new we are allocating dynamic memory for complex class objects .
- Objects get created on the heap section hence this object is called Heap based or dynamic object.
- Cptr is a complex type pointer which is holding the address of that dynamic object.



Difference between malloc and new

new is an operator.	malloc is a function.
new returns a typecasted Pointer, so no need to do explicit typecasting.	malloc returns void pointer, malloc returns void pointer, cast it explicitly, before use.
We must mention the datatype while allocating the memory with new.	malloc accepts only the exact no. of bytes required, so no need to mention datatype.
When memory is allocated with new, the constructor gets called for the object.	When memory gets allocated by malloc function, the constructor function does not get called.

Difference between malloc and new

new	malloc
Memory allocated by new is released by the operator delete.	Memory allocated by malloc is released by function free.
Destructor is called when memory is released with delete.	Destructor is NOT called when the memory is released for free.
To release memory for array syntax is delete[ ]	To release memory for array syntax is free(ptr);
In case new fails to allocate memory, it raises a Run time exception called as bad_alloc.	In case malloc fails to allocate memory it returns a NULL.

Sum function and Copy Constructor

- Copy constructor is a single parameter constructor hence it is considered as parameterized constructor
- Example: sum of two complex number

Complex sum(const Complex &c2)

Copy constructor

Complex c2(c1)

or

Complex c2=c1

C1 => 7 + j 6

C2 => 7 + j 6

Write a function in complex class to add 2 complex numbers

C1 => 7+j6

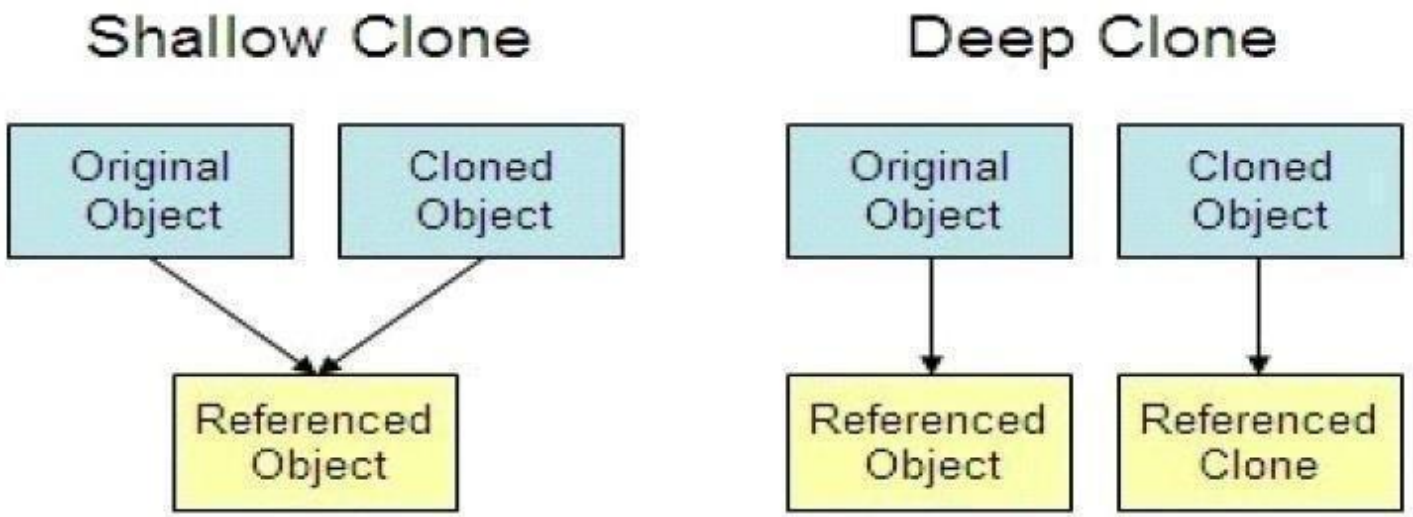
+

C2 => 3+j2

C3 => 10+j8

Object Copying

- In object-oriented programming, “object copying” is a process of creating a copy of an existing object.
- The resulting object is called an object copy or simply copy of the original object.
- Methods of copying:
  - Shallow copy
  - Deep copy



Types of Copy

The copy constructor is used to initialize the new object with the previously created object of the same class.

- **Shallow Copy**
  - The process of copying the state of an object into another object.
  - It is also called a bit-wise copy.
  - When we assign one object to another object at that time, we copy all the contents from source object to destination object as it is. Such a type of copy is called a shallow copy.
  - The compiler by default creates a shallow copy. Default copy constructors always create shallow copy.

- **Deep Copy**
  - Deep copy is the process of copying the state of the object by modifying some state.
  - It is also called a member-wise copy.
  - When class contains at least one data member of pointer type, and class contains user defined destructor then when we assign one object to another object then a copy constructor is called.
  - At that time instead of a copy base address, allocate a new memory for the newly created object and then copy contain from memory of the source object into memory of the destination object. This type of copy is called a deep copy.

Thank You

Thank You